

Super Resolution Reconstruction

Abstract: In the realm of digital image processing, enhancing image quality through a technique known as super-resolution (SR) is a critical and challenging task. This report delves into the application of advanced deep learning models, specifically SRResNet and SRGAN, to achieve high-resolution images from low-resolution inputs. SRResNet leverages the robustness of residual networks (ResNets) to enhance image accuracy during the upscaling process. Conversely, SRGAN employs a generative adversarial network (GAN) framework to ensure that upscaled images not only appear realistic but also possess the correct level of detail. This document will provide an in-depth analysis of the architectural design of these two models, the methodologies employed for their training, and a comprehensive evaluation of their performance in the context of digital image processing.

Content

1	Introduction.....	3
1.1	Definition of Super-resolution Reconstruction.....	3
1.2	Project Environment.....	3
2	Method	3
2.1	SRResNet.....	3
2.1.1	Network Architecture	4
2.1.2	Model setup	4
2.2	SRGAN	5
2.2.1	Network Architecture	6
2.2.2	Model setup	6
3	Model Training.....	8
3.1	Dataset.....	8
3.1.1	SRResNet	8
3.1.2	SRGAN.....	9
3.2	Loss Functions	10
3.2.1	SRResNet.....	10
3.2.2	SRGAN.....	11
3.3	Training Process	15
3.3.1	SRResNet.....	15

3.3.2	SRGAN ver1.....	18
3.3.3	SRGAN ver2.....	20
4	Evaluation & Results	23
4.1	Evaluation Metrics	23
4.2	Other Evaluation Methods	错误!未定义书签。
4.3	Result	24
5	Discoveries & Future Improvements	26
5.1	Discoveries.....	26
5.2	Future Improvements.....	27
6	Conclusion.....	28
	Reference	28

1 Introduction

1.1 Definition of Super-resolution Reconstruction

The super-resolution reconstruction of an image refers to the restoration of a given low-resolution image into a corresponding high-resolution image by a specific algorithm. The main concept is to turn a small-sized image into a large-size image to make the image more clear

In this paper will use both SRResNet and SRGAN to implement the super-resolution reconstruction where SRResNet enhances reconstruction accuracy by incorporating residual blocks and optimizing pixel-level mean squared error (MSE). On the other hand,*SRGAN uses a GAN framework to combine perceptual and adversarial losses, producing high-resolution images with naturalistic textures. This paper explores the theoretical foundations, network designs, and performance differences between these two methods.

1.2 Project Environment

GPU: NVIDIA GeForce 1650 -7.5(computing capacity);

CUDA Toolkit11ver; Cudnn: 8.6.0.163; Visual Studio 2022; matlab2018;

Python3.9; Pytorch; tensorflow

2 Method

2.1 SRResNet

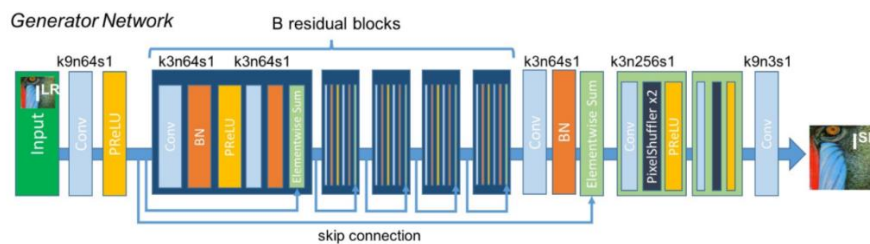
SRResNet Super-Resolution Method:

SRResNet is an image super-resolution method based on residual networks (ResNet). In the context of digital image processing, the goal of super-resolution reconstruction is to enhance the resolution of an image, ensuring that it remains clear and detailed when upscaled. The primary idea behind SRResNet is to use residual connections to mitigate the vanishing gradient problem, allowing the network to learn deeper and more complex mappings.

In digital image processing:

1. **Residual Learning:** SRResNet employs a residual learning framework by incorporating residual blocks, which include skip connections, allowing the network to directly learn the residual (the difference) between the low-resolution image and the high-resolution image. This approach helps maintain gradient flow during training, enabling the training of deeper network architectures.

2. **Mitigating the Vanishing Gradient Problem:** In deep learning, the vanishing gradient problem is common, especially in deep networks. SRResNet alleviates this issue with residual connections, allowing the network to train more layers without the gradients disappearing during backpropagation.
3. **Pixel-Level Mean Squared Error Optimization:** During training, SRResNet optimizes the pixel-level mean squared error (MSE), a loss function that measures the difference between the predicted image and the actual high-resolution image. By minimizing this error, the network learns how to reconstruct high-resolution images more accurately.
4. **Learning Complex Mappings:** Thanks to the introduction of residual connections, SRResNet can learn more complex mappings, which is particularly important in digital image processing as it involves recovering details and textures from low-resolution images to high-resolution images.



2.1.1 Network Architecture

Input and Output: The input is a low-resolution image, and the output is a high-resolution image.

Residual Blocks: Each residual block consists of two 3×3 convolutional layers, followed by batch normalization and a ReLU activation.

Upsampling Module: Pixel shuffle layers are used for upscaling the image resolution.

Loss Function: SRResNet minimizes the mean squared error (MSE) loss to optimize pixel-level accuracy.

2.1.2 Model setup

1. Convolutional Module:

- First convolutional layer: Utilizes a convolution kernel of size `large_kernel_size`, which is 9×9 , for the network's input part to extract features from the initial low-resolution image.
- Last convolutional layer: Similarly uses a convolution kernel of size `large_kernel_size`, which is 9×9 , typically for the network's output part to generate the high-resolution image.

2. Residual Block:

- Number: `n_blocks`, which is 16 residual blocks.
- Each convolutional layer within the residual blocks usually employs a `small_kernel_size` which is 3x3, to reduce the number of parameters and computational load while maintaining model depth.
- Number of channels in the intermediate layer: `n_channels`, which is 64 channels, indicating that each convolutional layer within the residual block will process 64 feature maps.

3. Sub-pixel Convolutional Module:

- Number: 2 sub-pixel convolutional modules, which are typically used to perform pixel shuffle operations, upsampling low-resolution feature maps to high-resolution through nonlinear transformations and rearrangement

4. Training Parameter Setup:

- `batch_size` = 128
- `start_epoch` = 1
- `epochs` = 130
- `workers` = 4
- `lr` = 1e-4

2.2 SRGAN

SRGAN (Super-Resolution Generative Adversarial Network) is the first framework to apply generative adversarial networks (GANs) to super-resolution tasks, enabling the generation of high-resolution images with realistic textures.

The core idea of SRGAN is to use the framework of generative adversarial networks and adversarial training to generate high-quality super-resolution images. SRGAN consists of two parts:

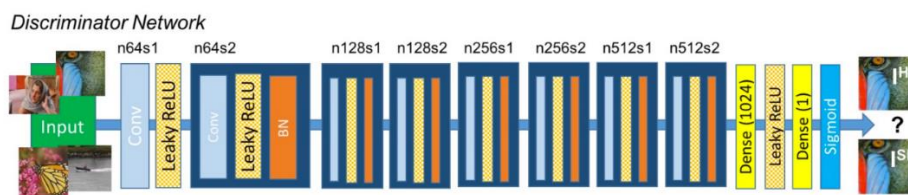
1. **Generator Network:** Receives low-resolution images as input and generates corresponding high-resolution images through a deep convolutional neural network.
2. **Discriminator Network:** Determines whether the input image is a real high-resolution image or a fake high-resolution image generated by the generator.

During the training process, the generator network continuously optimizes to generate more realistic high-resolution images, while the discriminator

network strives to improve its ability to distinguish between real and fake images. Through this adversarial training, SRGAN can ultimately generate super-resolution images with better visual effects.

Compared to traditional methods, SRGAN has the following advantages:

- It can recover more details and texture information.
- The generated images are more realistic and natural.
- It can achieve magnification factors of 4 times or higher.



2.2.1 Network Architecture

Generator: The generator is based on the SRResNet structure but incorporates perceptual and adversarial losses.

Discriminator: The discriminator is a convolutional network designed to distinguish between real and generated images.

2.2.2 Model setup

1. SRResNet Generator Parameters:

- I. First Convolutional Layer Kernel Size (`large_kernel_size_g`): 9
 - Used in the first convolutional module of the generator, which receives the low-resolution input image and begins the feature extraction process.
- II. Intermediate Convolutional Layer Kernel Size (`small_kernel_size_g`): 3
 - Used in the 16 residual modules of the generator, which further extract and process features while maintaining the flow of information.
- III. Intermediate Layer Channel Number (`n_channels_g`): 64
 - The number of channels in the residual modules, determining the depth of the feature maps.
- IV. Number of Residual Modules (`n_blocks_g`): 16
 - The number of residual modules included in the generator, which helps train deep networks and prevent the vanishing gradient problem.

- V. Last Convolutional Layer Kernel Size (`large_kernel_size_g`): 9
 - Used in the last convolutional module of the generator, which maps the extracted features back to a high-resolution image.
 - VI. Sub-Pixel Convolutional Layers:
 - The generator includes 2 sub-pixel convolutional modules, which use smaller convolutional kernels (usually 3x3) to further refine the feature maps to generate higher-resolution images.
 - VII. Pre-trained Model Path (`srresnet_checkpoint`):
 - The file path of the pre-trained SRResNet model, used to initialize the weights of the generator for faster convergence and improved training efficiency.
2. Discriminator Parameters:
- I. Convolutional Kernel Size (`kernel_size_d`): 3
 - Used in all convolutional modules of the discriminator, which extracts features from the input image and judges whether it is a high-resolution image.
 - II. First Convolutional Module Channel Number (`n_channels_d`): 64
 - The number of channels in the first convolutional module of the discriminator, with the channel number doubling every other module to enhance the expression of features.
 - III. Number of Convolutional Modules (`n_blocks_d`): 8
 - The number of convolutional modules included in the discriminator, used to gradually extract and process features.
 - IV. Fully Connected Layer Connections (`fc_size_d`): 1024
 - The number of connections in the fully connected layer of the discriminator, to determine whether the input image is a real high-resolution image or one generated by the generator.

3. Training Parameter Setup:

Version1

- `batch_size` = 128
- `start_epoch` = 1
- `epochs` = 100
- `checkpoint` = None (SRGAN – pretraining model)
- `workers` = 4
- `vgg19_i` = 5 (VGG19 network's i-th pooling layer)

- vgg19_j = 4 (VGG19 network's j-th convolutional layer)
- beta = 1e-3
- lr = 1e-4

result: it gets a error of gpu is out of memory, the batch size is too big to run.

Version2

- batch_size = 45
- start_epoch = 1
- epochs = 100
- checkpoint = None (SRGAN – pretraining model)
- workers = 4
- vgg19_i = 5 (VGG19 network's i-th pooling layer)
- vgg19_j = 4 (VGG19 network's j-th convolutional layer)
- beta = 1e-3
- lr = 1e-4

result: the largest batch size that the program can run

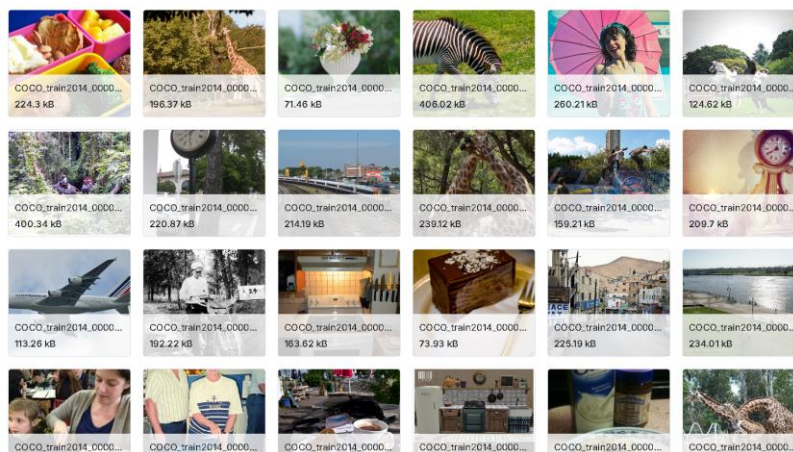
3 Model Training

3.1 Dataset

3.1.1 SRResNet

Training set: COCO2014 (include: train2014, val2014)

Testing set: Set14 Set5 BSD100




```
正在创建文件列表... 请耐心等待.
训练集中共有 123285 张图像
在测试集 BSD100 中共有 100 张图像
在测试集 Set5 中共有 5 张图像
在测试集 Set14 中共有 14 张图像
生成完毕。训练集和测试集文件列表已保存在 ./data/ 下
```

3.1.2 SRGAN

Version1

Training set: COCO2014 (include: train2014, val2014)

Testing set: Set14 Set5 BSD100

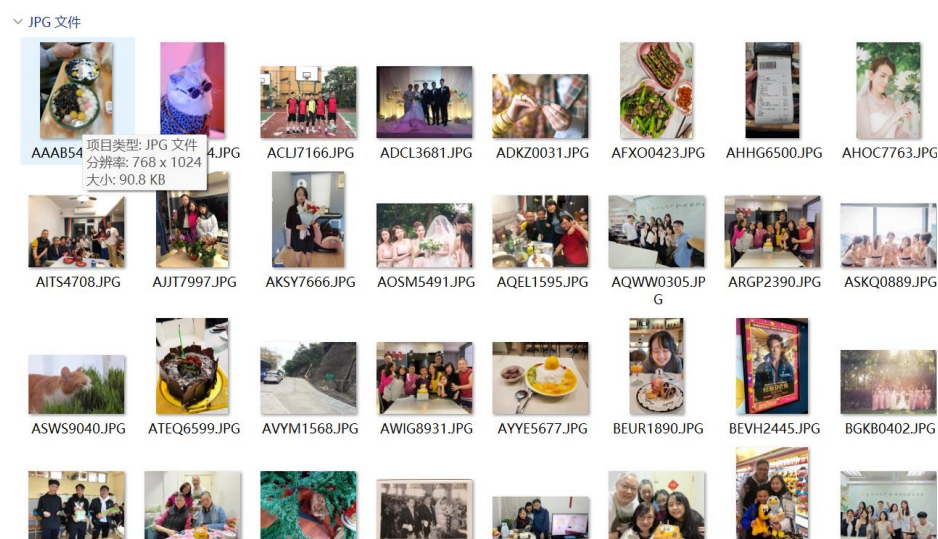
Result: cannot run, gpu is out of error

Version2

Training set: mydata

Testing set: Set14 Set5 BSD100

Result: it can run properly, however the the traning data is not enough to train a good observation.



```
训练集中共有 41115 张图像
在测试集 BSD100 中共有 100 张图像
在测试集 Set5 中共有 5 张图像
在测试集 Set14 中共有 14 张图像
生成完毕。训练集和测试集文件列表已保存在 ./mydata/ 下
```

Version3

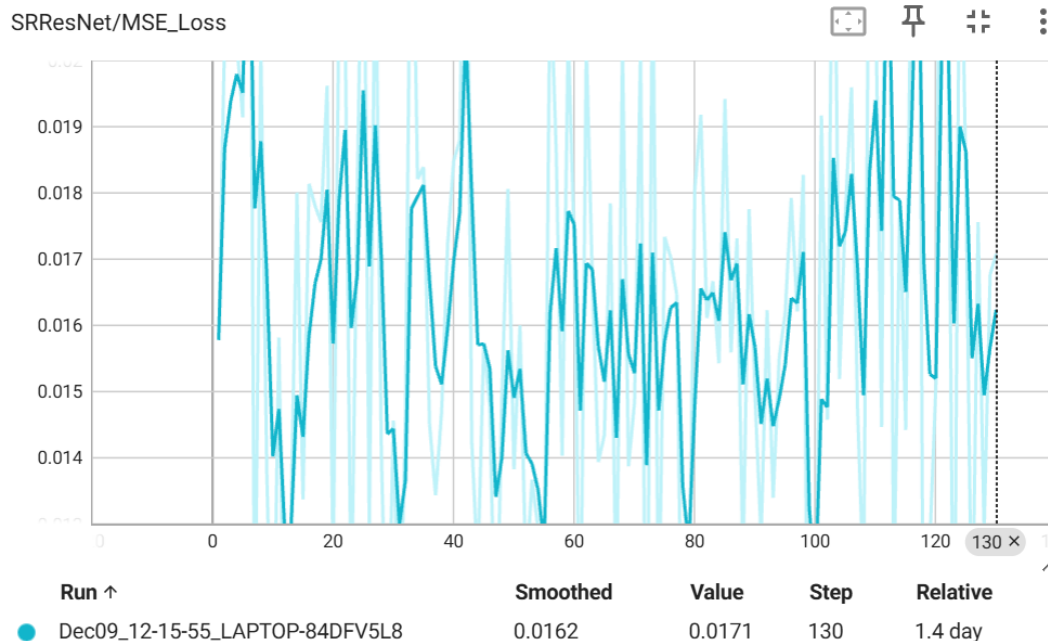
Training set: mydata . COCO2014 (only:val2014)

Testing set: Set14 Set5 BSD100

Result: beside “mydata”set I add COCO2014 (only:val2014) also, then start training. The observation I get is more better than the pervious version (the proccess will shown at 3.3.) and the obervation get a good result than SRResNet, but the different is not too clear.

3.2 Loss Functions

3.2.1 SRResNet



Above line graph illustrates the changes in Mean Squared Error (MSE) loss values of the SRResNet model during the training process.

1. **Fluctuation in Loss Values:** The undulations in the curve reflect the changes in loss values throughout the training. Ideally, as training progresses, the loss values should gradually decrease, indicating that the model is learning and improving its performance.
2. **Initial Phase:** During the early stages of training, there may be significant fluctuations in the loss values. This is because the model is just beginning to learn, with substantial adjustments to the weights, leading to sharp changes in the loss values.
3. **Convergence Trend:** As training continues, if the curve starts to level off, this may indicate that the model is converging, meaning the learning process is stabilizing, and the adjustments to the weights are becoming smaller.
4. **Signs of Overfitting:** If the loss values continue to decrease on the training set but begin to rise on the validation set, this could be a sign

of overfitting, where the model performs well on the training data but poorly on unseen data.

5. **Smoothed Value:** The graph shows a smoothed value of 0.0162, which is likely calculated using a moving average or other smoothing techniques to reduce noise and more clearly display the overall trend of the loss values.
6. **Final Value:** The last recorded loss value is 0.0171, which is the loss value at step 130.
7. **Training Duration:** The relative time is shown as 1.4 days, which likely refers to the time taken to reach step 130 in the training process.
8. **Number of Training Steps:** The steps on the X-axis represent the number of iterations during the training process. In this graph, the training has gone through 130 steps.
9. **Training Stability:** The stability of the curve can reflect the stability of the training process. If the curve remains very volatile in the later stages, adjustments to the learning rate or other hyperparameters may be necessary.
10. **End of Training:** The vertical dashed line in the graph may indicate the end of the training or a significant checkpoint.

3.2.2 SRGAN

For SRGAN there have three loss function

Generation Loss (Generation Loss): This loss function is typically used to measure the difference between the images generated by the generator and the target images. In SRGAN, generation loss may include both adversarial loss and content loss. Adversarial loss encourages the generator to produce images that can deceive the discriminator, making it unable to distinguish between generated images and real images. Content loss ensures that the generated images are similar in content to the target images, often calculated by comparing the feature space representations of the generated and target images.

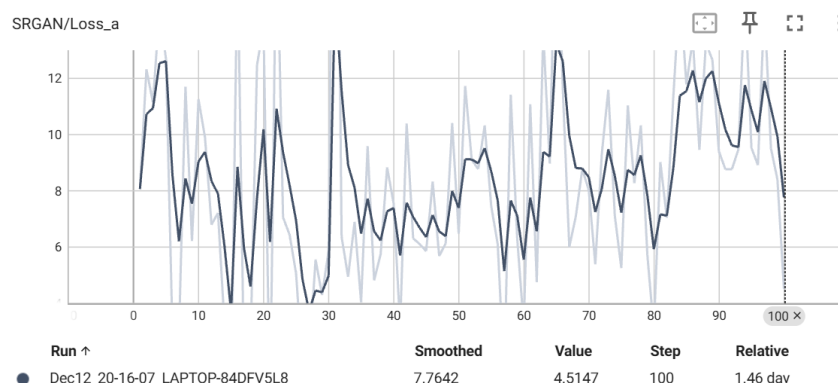
Content Loss (Content Loss): Content loss is used to ensure that the generated high-resolution images are similar in content to the original high-resolution images. This is typically achieved by comparing the feature maps of the generated images and the target images using a pre-trained model like the VGG network. Content loss helps to preserve the texture and details of the images, making the generated images more visually realistic.

Discriminator Loss (Discriminator Loss): The discriminator loss is used to train the discriminator to differentiate between real and generated images. The discriminator aims to maximize its ability to correctly classify images as either real or generated. In SRGAN, the discriminator loss may include classification losses for both real and generated images, often calculated using binary cross-entropy. The goal of the discriminator is to accurately identify real images and reject the outputs of the generator.

Same feature of both three loss function:

1. **Fluctuation in Loss Values:** The undulations in the curve reflect the changes in loss values throughout the training. Fluctuations may indicate varying effectiveness in how the model learns from the data at different training steps.
2. **Initial Phase:** During the early stages of training, there may be significant fluctuations in the loss values. This is likely because the model is still learning how to generate high-quality super-resolved images.
3. **Number of Training Steps:** The steps on the X-axis represent the number of iterations during the training process. In this graph, the training has gone through 100 steps.
4. **Training Stability:** The stability of the curve can reflect the stability of the training process. If the curve remains very volatile in the later stages, adjustments to the learning rate or other hyperparameters may be necessary.

Generation loss



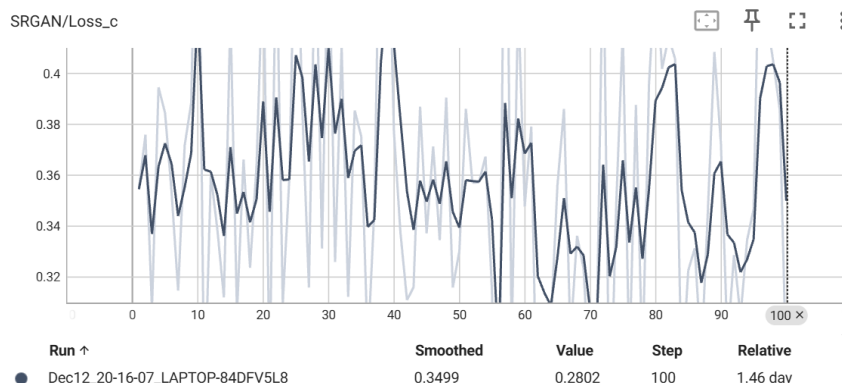
Above line graph illustrates the generation loss of a Super-Resolution Generative Adversarial Network (SRGAN) model during its training process

1. **Convergence Trend:** If the curve begins to stabilize in the later stages of training, this could suggest that the model is converging, meaning

the learning process is becoming more stable and the quality of the images generated by the generator is improving. And we can see the graph is kind of unstable.

2. **Signs of Overfitting:** If the loss values continue to decrease on the training set but start to increase on the validation set, this could be a sign of overfitting, where the model performs well on the training data but poorly on unseen data.
3. **Smoothed Value:** The graph shows a smoothed value of 7.7642, which is likely calculated using a moving average or other smoothing techniques to reduce noise and more clearly display the overall trend of the loss values.
4. **Final Value:** The last recorded loss value is 4.5147, which is the loss value at step 100.
5. **Training Duration:** The relative time is shown as 1.46 days, which likely refers to the time taken to reach step 100 in the training process.

Content loss



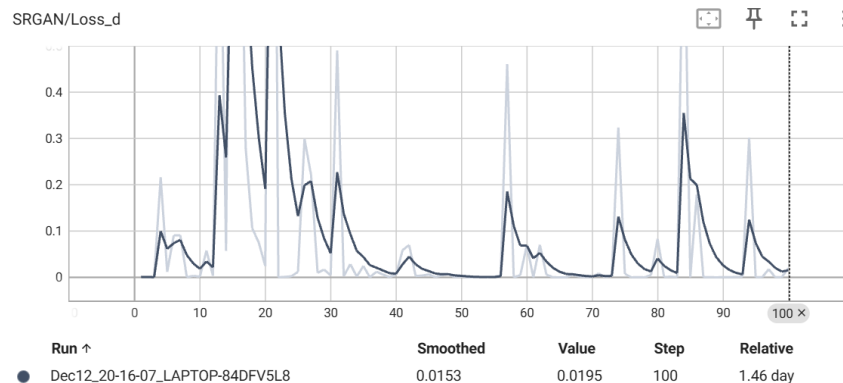
Above line graph depicts the content loss of an SRGAN (Super-Resolution Generative Adversarial Network) model during its training process

1. **Convergence Trend:** If the curve becomes more stable in the later stages of training, this could indicate that the model is converging, meaning the learning process is stabilizing, and the generator is producing images that are increasingly similar to the target images in content. And we can see the graph is kind of unstable.
2. **Signs of Overfitting:** If the content loss continues to decrease on the training set but begins to rise on the validation set, this could be a sign of overfitting, where the model performs well on the training data but poorly on unseen data.
3. **Smoothed Value:** The graph shows a smoothed value of 0.3499, which is likely calculated using a moving average or other smoothing

techniques to reduce noise and more clearly display the overall trend of the loss values.

4. **Final Value:** The last recorded content loss value is 0.2802, which is the loss value at step 100.
5. **Training Duration:** The relative time is shown as 1.46 days, which likely refers to the time taken to reach step 100 in the training process.

Discriminant loss



This line graph represents the discriminator loss of an SRGAN (Super-Resolution Generative Adversarial Network) model during its training process. The discriminator's role in a Generative Adversarial Network (GAN) is to distinguish between real images and those produced by the generator. Here are some key points for interpreting this graph:

1. **Convergence Trend:** If the curve tends to stabilize in the later stages of training, this could suggest that the discriminator is converging, meaning the learning process is becoming more stable, and the discriminator's ability to discern images is improving. And we can see that compare to previous two graph this loss function is Relatively stable.
2. **Pattern Recognition:** The periodic fluctuations in the discriminator loss may indicate the adversarial process between the generator and the discriminator. At certain steps, the generator might produce images that are good enough to deceive the discriminator, leading to a decrease in loss.
3. **Smoothed Value:** The graph shows a smoothed value of 0.0153, which is likely calculated using a moving average or other smoothing techniques to reduce noise and more clearly display the overall trend of the loss values.
4. **Final Value:** The last recorded discriminator loss value is 0.0195, which is the loss value at step 100.

5. **Training Duration:** The relative time is shown as 1.46 days, which likely refers to the time taken to reach step 100 in the training process.

3.3 Training Process

For all provided image, The first line (?_1) is a low-resolution image, the second line (?_2) is a super-resolution image reconstructed by the current model, and the third line (?_3) is a real original clear image of the actual reality.

3.3.1 SRResNet

Epoch 1 (start)

SRResNet/epoch_1_1



Dec09_12-15-55_L...

Step 1



SRResNet/epoch_1_2



Dec09_12-15-55_L...

Step 1



SRResNet/epoch_1_3



Dec09_12-15-55_L...

Step 1



Comparing the low-resolution and the original (high-resolution) images, after the first training and reconstruction, the boundaries of the image have an outline but it is **not** very distinct, and the texture is basically not recognizable.

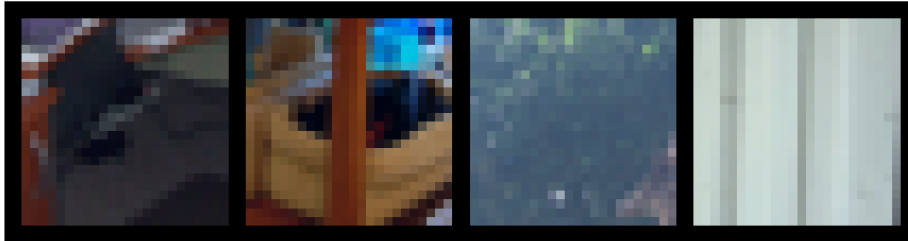
Epoch 81 (middle)

SRResNet/epoch_81_1

Dec09_12-15-55_L...



Step 81



SRResNet/epoch_81_2

Dec09_12-15-55_L...



Step 81



SRResNet/epoch_81_3

Dec09_12-15-55_L...



Step 81



Comparing the low-resolution image to the original high-resolution image in SRResNet, after the 81st training iteration, the reconstructed image has **more** clear boundaries, but the texture is still not discernible.

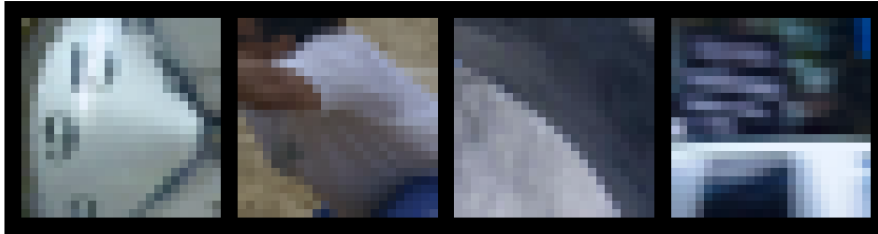
Epoch 130 (final)

SRResNet/epoch_130_1

Dec09_12-15-55_L...



Step 130



SRResNet/epoch_130_2

Dec09_12-15-55_L...



Step 130



SRResNet/epoch_130_3

Dec09_12-15-55_L...



Step 130

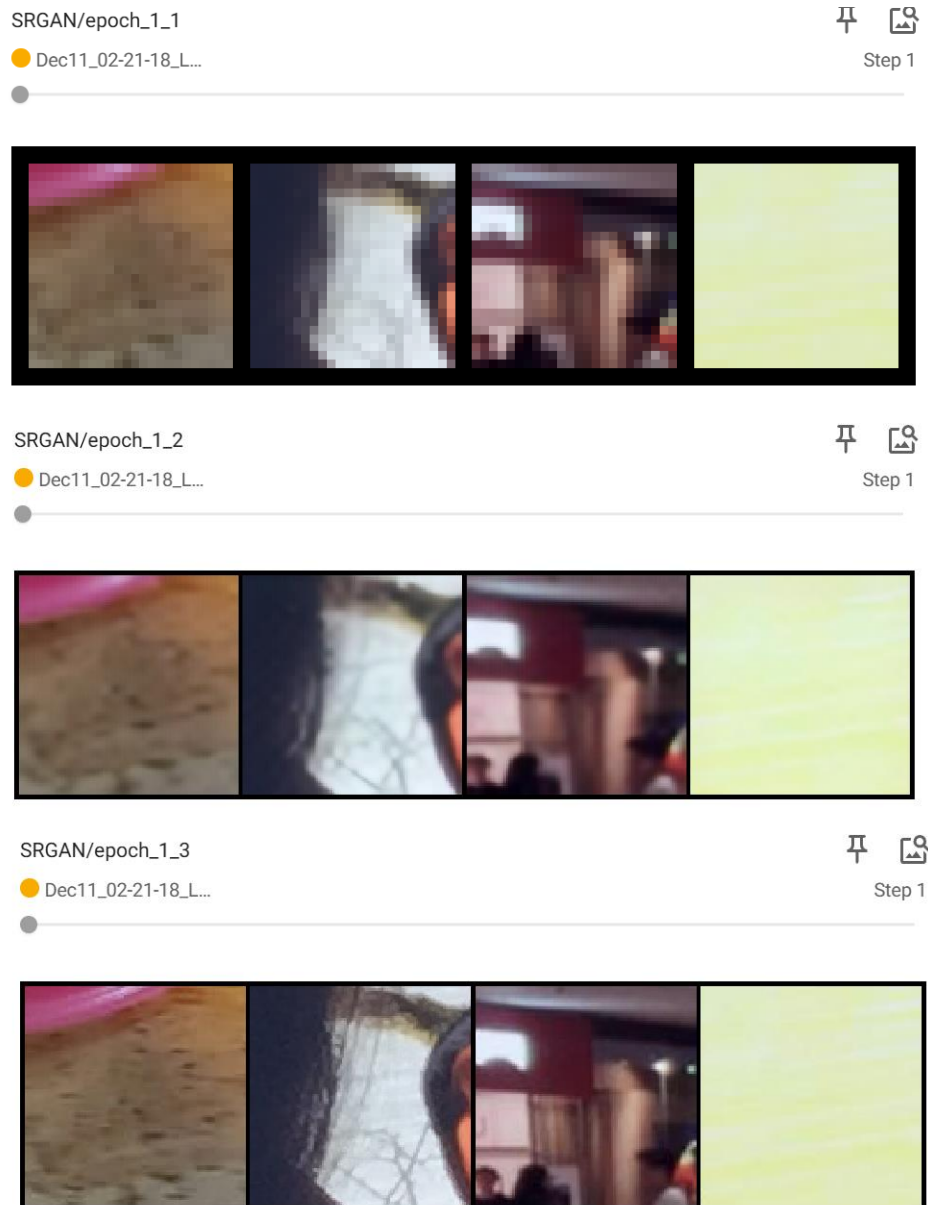


Comparing the low-resolution image to the original high-resolution image in SRResnet, after the 130th training iteration, the reconstructed image has **most** clear boundaries, but the texture is still not discernible.

We suppose that SRGAN should have a better result than SRResNet, and both SRGAN version 1 and SRGAN version 2 used the same parameter but version 1 use data set of “mydata” and version 2 use both “mydata”, “COCO2014 (only:val2014)”

3.3.2 SRGAN ver1

Epoch 1



Comparing the low-resolution image to the original high-resolution image in SRGAN, after the 1st training iteration, the reconstructed image has **most** clear boundaries, but the texture is still not discernible.

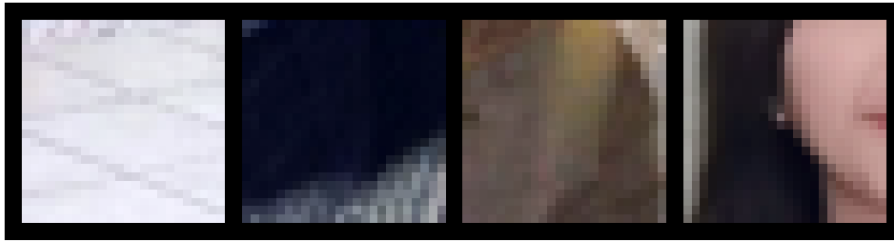
Epoch 47

SRGAN/epoch_47_1

● Dec11_02-21-18_L...



Step 47



SRGAN/epoch_47_2

● Dec11_02-21-18_L...



Step 47



SRGAN/epoch_47_3

● Dec11_02-21-18_L...



Step 47

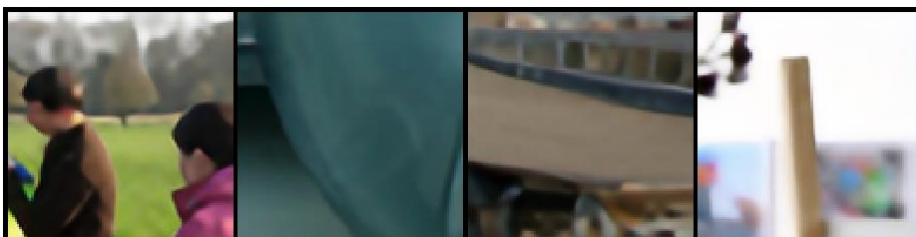


SRResNet/epoch_47_2

● Dec09_12-15-55_L...



Step 47

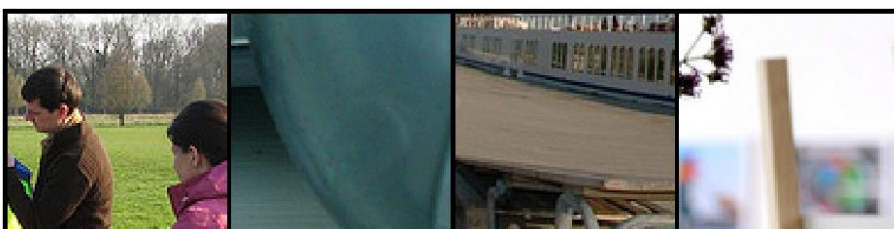


SRResNet/epoch_47_3

● SRResNet/epoch_47_3



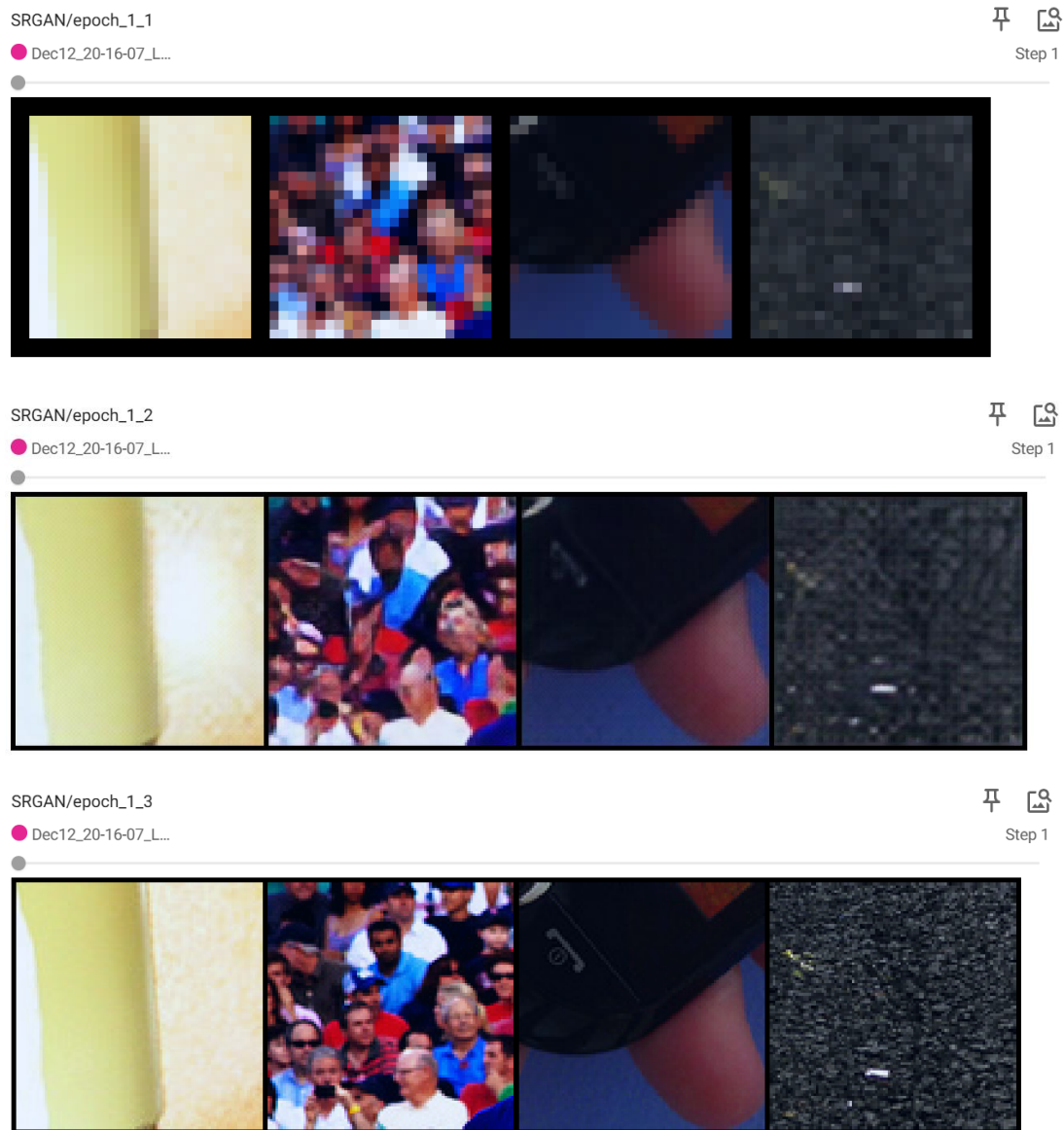
Step 47



Above are 5 images that shows the 47th training result of SRGAN and SRResNet, we can see in the same epoch, SRGAN generates a better image than SRResNet

Comparing the low-resolution image and the original high-resolution image, the reconstructed image are both clear boundaries. On the other hand, Some of the texture can be clearly seen in SRGAN (seen in the red circle) and image are still not discernible in SRResNet.

3.3.3 SRGAN ver2



Comparing the low-resolution image to the original high-resolution image in SRGAN, after the 1st training iteration, the reconstructed image has **most** clear boundaries, Some of the texture can be seen.

Epoch 50

SRGAN/epoch_50_1

● Dec12_20-16-07_L...

🔍 🖼
Step 50



SRGAN/epoch_50_2

● Dec12_20-16-07_L...

🔍 🖼
Step 50



SRGAN/epoch_50_3

● Dec12_20-16-07_L...

🔍 🖼
Step 50



Comparing the low-resolution image to the original high-resolution image in SRGAN, after the 1st training iteration, the reconstructed image has **most** clear boundaries, Some of the texture can be clearly seen.

Epoch 100

SRGAN/epoch_100_1

● Dec12_20-16-07_L...



Step 100



SRGAN/epoch_100_2

● Dec12_20-16-07_L...



Step 100



SRGAN/epoch_100_3

● Dec12_20-16-07_L...



Step 100

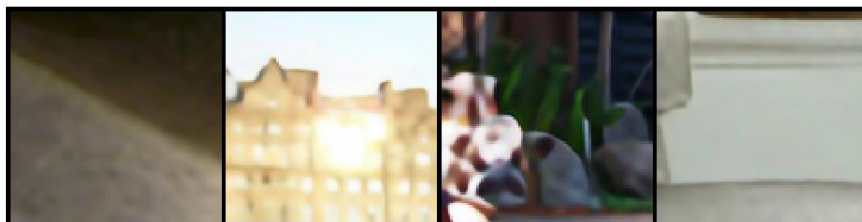


SRResNet/epoch_100_2

● Dec09_12-15-55_L...



Step 100



SRResNet/epoch_100_3

● Dec09_12-15-55_L...



Step 100



Above are 5 images that shows the 100th training result of SRGAN and SRResNet, we can see in the same epoch, SRGAN generates a better image than SRResNet, Comparing the low-resolution image and the original high-resolution image, the reconstructed image are both clear boundaries. On the other hand, many of the texture can be clearly seen (seen in the red circle) and image are still not discernible in SRResNet.

4 Evaluation & Results

We are going to evaluate the last version of the training.

4.1 Evaluation Metrics

We use PSNR and SSIM to evaluate the model.

For PSNR, The minimum PSNR is 0, and the larger the PSNR, the smaller the difference between the two images. For SSIM, $SSIM \leq 1$, the larger the SSIM, the more similar the two images are.

```
数据集 Set5:
SRGAN 评估结果 :
PSNR  29.224
SSIM  0.847
平均单张样本用时  3.552 秒

数据集 Set14:
SRGAN 评估结果 :
PSNR  25.718
SSIM  0.698
平均单张样本用时  1.174 秒

数据集 BSD100:
SRGAN 评估结果 :
PSNR  25.037
SSIM  0.659
平均单张样本用时  0.535 秒
```

Fig: SRGAN

```
数据集 Set5:
SRRESNET 评估结果 :
PSNR  31.909
SSIM  0.901
平均单张样本用时  2.313 秒

数据集 Set14:
SRRESNET 评估结果 :
PSNR  28.571
SSIM  0.797
平均单张样本用时  1.132 秒

数据集 BSD100:
SRRESNET 评估结果 :
PSNR  27.521
SSIM  0.753
平均单张样本用时  0.528 秒
```

Fig: SRResNet

In my evaluation, we can see that the value of SRResNet is bigger than SRGAN, it shows that SRResNet have a better generate result than SRGAN. Normally, We suppose that SRGAN should have a better result than SRResNet. The reason why gets this result the possible reason is because I had lower the parameter and had reduce the number of picture.

However what we had got in the observation, we can also know that SRGAN can have a better result than SRResNet.

4.2 Result



Original



Bicubic



SRGAN



SRResNet



Original



Bicubic



SRResNet



SRGAN

For thos two image, I generate 3 situation (include: Bicubic, SRResNet , SRGAN),we can see that after the method of Bicubic, image become more clear than the original one but there still have noise in the image. Then after SRResNet, it reduce the noise. And after generate by SRGAN it become more clear and noise reduce again.

5 Discoveries & Future Improvements

5.1 Discoveries

1. SRResNet Training Progress:

- By the 81st training iteration of SRResNet, the model has started to produce images with clear boundaries but has not yet captured fine texture details.
- This suggests that SRResNet may prioritize edge sharpness over texture restoration in the early stages of training.

2. Comparison between SRGAN and SRResNet:

- At the 100th training iteration, SRGAN outperforms SRResNet in terms of both boundary clarity and texture detail, as indicated by the clear textures in the red circle area.
- This implies that SRGAN, which is a generative adversarial network, might be better suited for restoring complex textures and details compared to SRResNet, which is a more straightforward residual network.

3. Evaluation Metrics Discrepancy:

- Despite the visual superiority of SRGAN's results, the evaluation metrics suggest that SRResNet has a higher value, indicating a better generation result according to those metrics.
- This discrepancy could be due to the metrics not fully capturing the quality of texture details or the subjective nature of visual perception versus objective measurement.

4. Training Parameters and Data Quantity:

- The commenter suggests that lowering the parameters and reducing the number of training images might have affected the performance of SRGAN, potentially leading to the unexpected result where SRResNet had a higher metric value.
- This indicates that the choice of hyperparameters and the quantity of training data can significantly impact the performance of deep learning models.

5. GPU Limitations:

- The training speed is limited by the use of a single GPU, which can be a bottleneck for deep learning tasks that require extensive computational resources.

5.2 Future Improvements

1. Data Augmentation and Preprocessing:

- Expand the training dataset and use data augmentation techniques to improve the model's generalization capabilities.
- Optimize the image preprocessing steps to ensure that the input low-resolution images retain sufficient texture information for the model to learn from.

2. Hyperparameter Tuning:

- Fine-tune the model's hyperparameters, including learning rate, batch size, number of iterations, etc., to find the best configuration.

3. GPU:

- The mention of NVLink suggests that the use of multiple GPUs connected with NVLink could enhance training speed and potentially improve the model's performance by allowing for larger batch sizes or more complex models.
- Use multiple GPUs directly without NVLink

4. NVLink Technology:

- The support of NVLink in GPUs is highlighted as a capability that can meet the demands of advanced visual computing workloads.
- This implies that using NVLink to connect GPUs can be a beneficial strategy for improving the efficiency and effectiveness of deep learning models, particularly for tasks like super-resolution that are computationally intensive.

6 Conclusion

1. SRResNet focuses on pixel-level accuracy and is effective in reconstructing clear boundaries in images. However, it struggles to recover fine textures, resulting in images that lack detailed texture information.
2. SRGAN, which incorporates adversarial and perceptual losses, not only reconstructs clear boundaries but also captures more texture details compared to SRResNet. This makes the generated images appear more natural and visually appealing.
3. Training with a larger and more diverse leads to better overall performance in terms of both boundary clarity and texture detail.
4. Despite the limitations in texture recovery, SRResNet still provides a reasonable level of detail for many applications. However, for applications requiring high texture fidelity, SRGAN is the preferred choice.

Reference

SRGAN 综述 | Jan's Note |

<https://tokenjan.github.io/%E8%AE%BA%E6%96%87/2018/09/09/SRGAN/>

Set 5 & 14 Super Resolution Dataset |

<https://www.kaggle.com/datasets/ll01dm/set-5-14-super-resolution-dataset>

COCO2014 | <https://www.kaggle.com/datasets/nadaibrahim/coco2014>

BSD100 | <https://www.kaggle.com/datasets/huaiyingu/bsd100>

11 种图像清晰度评价函数 |

https://blog.csdn.net/kungfu_rabbit/article/details/90243838

残差网络 | https://blog.csdn.net/weixin_43135178/article/details/115447031

VGG19 模型 | <https://blog.csdn.net/u012739527/article/details/124012095>

SRCNN 到 EDSR | <https://blog.csdn.net/u013185349/article/details/84327998>

ML 重要概念 | <https://blog.csdn.net/walilk/article/details/50978864>

oneDNN | <https://zhuanlan.zhihu.com/p/483936837>